

# 1. Why are we using LSTM as the main model?

First understand the nature of our problem.

We have:

```
Day 1 → Day 2 → Day 3 → ... → Day 60 → Day 61
```

The **order of the data matters**.

We want the model to learn patterns such as:

"What happened during the previous several days may contain information about tomorrow."

This is a **time-series forecasting problem**.

LSTM is specifically designed for sequential data and has a memory mechanism that helps it retain useful information across time.

## 2. Why not simple models like Naive, ARIMA, SARIMA, ARIMAX, Exponential Smoothing?

### 2.1 Naive forecasting

The simplest possible idea:

Tomorrow's price = today's price.

Example:

```
Today's Close = ₹100

Naive prediction:
Tomorrow = ₹100
```

Your PRD explicitly defines this as the required naive baseline.

## 3. Why not ARIMA?

ARIMA stands for:

**AutoRegressive Integrated Moving Average**

It uses historical values and errors to model a time series.

Very simplified:



ARIMA can be useful for relatively structured time-series behavior.

But our project wants to investigate whether an **LSTM can learn nonlinear temporal patterns** from multiple features such as:

```
OHLCV
SMA20
SMA50
```

```
RSI
MACD
```

ARIMA in its basic form is not designed like a neural network to directly learn complex nonlinear relationships among all these sequential features.

---

## 4. Why not SARIMA?

SARIMA = Seasonal ARIMA.

It adds explicit **seasonality** to ARIMA.

For example:

```
Monthly sales
↓
Strong yearly seasonal pattern
```

SARIMA is very useful when there is a clear repeating seasonal structure.

For daily stock prices, seasonality is not the primary reason we are modelling the series.

So it isn't our main model.

---

## 5. Why not ARIMAX?

ARIMAX = ARIMA + external variables.

For example:

```
Stock price
+
Interest rate
+
Inflation
+
GDP
+
Oil price
```

It can be useful when you deliberately want external explanatory variables.

But our MVP primarily uses historical market data:

```
OHLCV
+
Technical indicators
```

and keeps external information such as news, macroeconomic events and fundamentals outside the mandatory MVP.

---

## 6. Why not Single Exponential Smoothing?

Exponential smoothing gives more importance to recent observations.

Very simplified:

```
Recent observation → more weight
Old observation → less weight
```

It is simple and useful for certain forecasting problems.

But again, our goal is to investigate a sequential deep-learning model using multiple features.

---

## 7. Why is Naive our baseline?

Because it is:

- extremely simple
- fast
- easy to understand
- difficult to accidentally implement incorrectly
- a meaningful reference point

Suppose:

```
Naive RMSE = 10
LSTM RMSE = 8
```

Now we know:

| LSTM improved over a very simple strategy.

But:

```
Naive RMSE = 8
LSTM RMSE = 10
```

Then:

| Our complicated LSTM didn't beat the simple baseline.

That is valuable information.

---

## 9. Why not use ARIMA as the baseline instead?

You **can** use ARIMA as an additional baseline.

In fact, a stronger experimental comparison could be:

```
Model 1 → Naive
Model 2 → Moving Average
Model 3 → ARIMA
Model 4 → LSTM
```

But your PRD requires **at least one simple baseline**, and specifically defines the naive baseline.

So:

### Required

```
Naive
```

### Optional stronger comparison

```
Naive
+
Moving Average
+
ARIMA
+
LSTM
```

This would make your research comparison stronger if you have enough time.

---

## 10. What does "naive" mean?

Naive = very simple assumption.

In our case:

The best guess for tomorrow's price is today's price.

Mathematically:

$$\hat{y}_{t+1} = y_t$$

Where:

- $y_t$  = today's actual closing price
- $\hat{y}_{t+1}$  = predicted tomorrow's price

## 11. What does "bias" mean?

If by **bias** you mean the ML/statistical term:

Bias is the error caused by a model being too simple and making strong assumptions.

Example:

A model always predicts:

₹100

even when the actual values are:

₹90  
₹110  
₹120  
₹80

It may have high bias because it is too simplistic.

### Bias vs variance

High Bias  
↓  
Model too simple  
↓  
Underfitting

High Variance  
↓  
Model too sensitive to training data  
↓  
Overfitting

The **naive model is intentionally simple**. We use it as a benchmark, not because we expect it to understand the market deeply.

## 12. What about Naive Bayes?

Naive Bayes is a **classification algorithm** based on Bayes' theorem.

It isn't our natural baseline for predicting a continuous stock closing price.

Our target is:

₹523.45

which is a continuous number.

So our naive price baseline is much more appropriate.

---

## 13. What is LSTM?

LSTM =

Long Short-Term Memory

It is a type of recurrent neural network designed to process sequential data.

Think of it like a smart memory system.

Suppose we give:

```
Day 1
Day 2
Day 3
...
Day 60
```

The LSTM tries to learn:

Which previous information is useful for predicting the next day?

---

## 14. Why does LSTM have gates?

LSTM has **three main gates**:

```
1. Forget Gate
2. Input Gate
3. Output Gate
```

And it also maintains:

```
Cell State
Hidden State
```

---

## 15. Forget Gate

Question:

"What old information should I forget?"

Example:

Suppose a stock had an unusual event 50 days ago.

The model may decide:

"That information is no longer useful."

The forget gate controls this.

Simple meaning:

```
Forget Gate
↓
What old information should remain?
```

# 16. Input Gate

Question:

"What new information should I remember?"

For example:

```
Today's price increased
Volume increased
RSI changed
MACD changed
```

The input gate determines which new information should be stored.

# 17. Output Gate

Question:

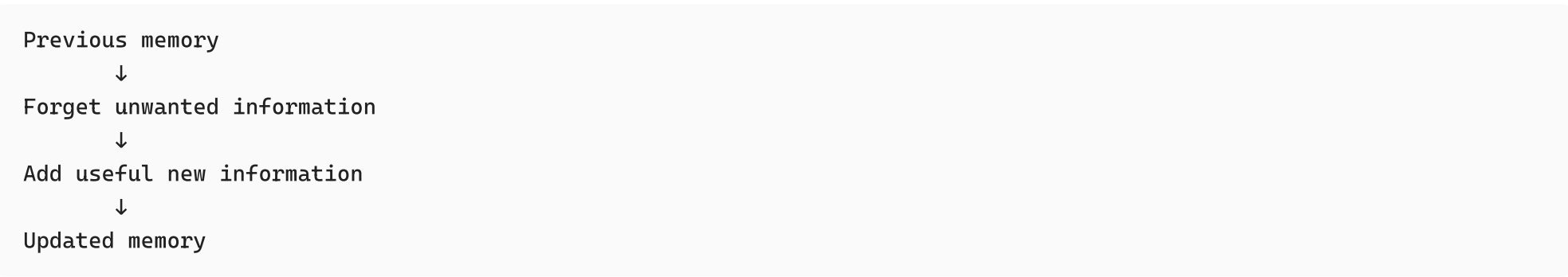
"What information should I output now?"

It determines what part of the cell's memory becomes the current hidden state.

# 18. Cell State

This is one of the most important concepts.

Think of the **cell state as the LSTM's long-term memory**.



The cell state is:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

where:

- $C_{t-1}$  = previous memory
- $f_t$  = forget gate
- $i_t$  = input gate
- $\tilde{C}_t$  = candidate new information

# 19. Hidden State

Hidden state is more like the **current output/short-term representation** of what the LSTM currently knows.

$$h_t = o_t * \tanh(C_t)$$

Think:

|                                     |
|-------------------------------------|
| Cell State                          |
| = long-term memory                  |
| Hidden State                        |
| = current useful information/output |

## 20. How are these actually calculated in our project?

You don't manually calculate them.

PyTorch's LSTM performs these calculations.

Conceptually:

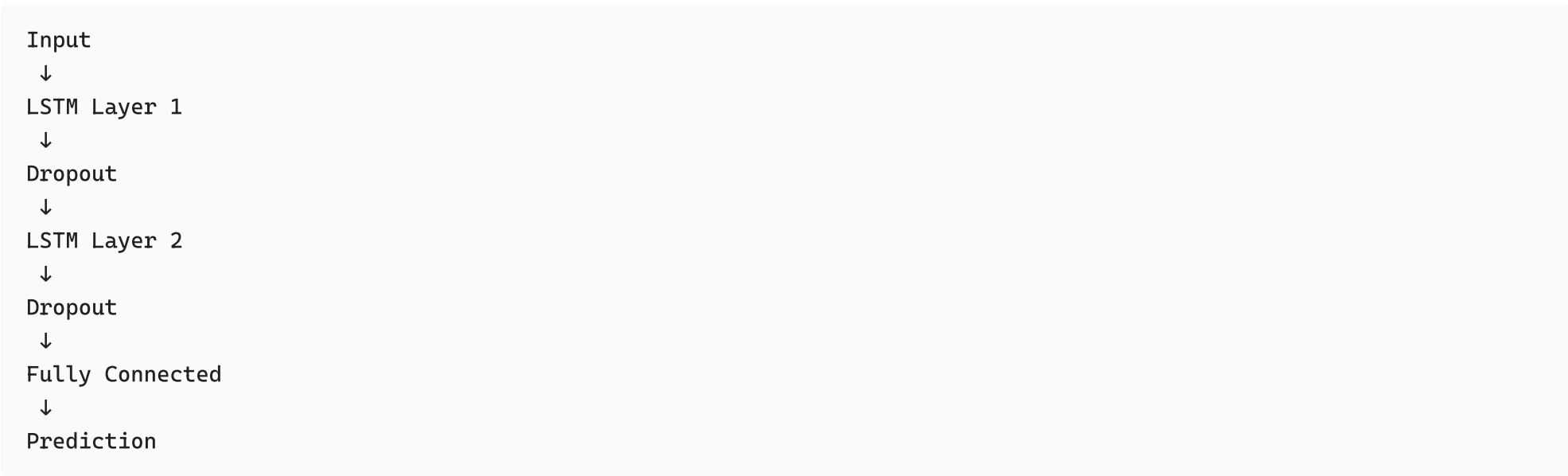


This happens for every time step in our 60-day sequence.

---

## 21. Why 2 LSTM layers?

Your initial architecture is:



Your PRD specifies 2 layers, hidden size 128 and dropout 0.2 as the initial configuration.

### Why 2?

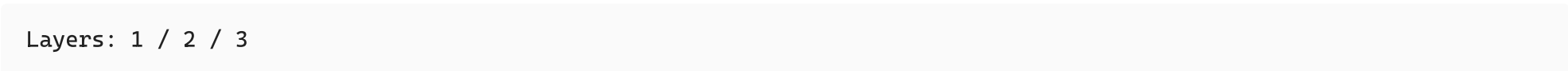
One layer can learn basic temporal representations.

A second layer can learn **higher-level temporal patterns from the representation produced by the first layer**.

Think:



We experiment with:



and:

```
Sequence: 30 / 60 / 90
Hidden size: 64 / 128 / 256
Dropout: 0.1 / 0.2 / 0.3
```

during hyperparameter tuning.

"Two layers are our initial configuration; we can validate this through hyperparameter tuning."

## 22. Why 60 days?

We will experiment with

```
30, 60 , 90
```

as candidates.

We choose **60 initially** because it gives the model a reasonably long recent historical context without making the sequence unnecessarily long.

Approximately:

```
60 trading days
≈ 3 months of market sessions
```

It gives the model information about:

- recent trend
- short-term volatility
- recent momentum
- volume behavior
- indicator patterns

**But:**

Maybe 30 works better.

Maybe 90 works better.

We don't assume.

We test.

## 23. What is OHLCV?

**O = Opening price.**

**H = High [Highest price during the day.]**

**L = Low [Lowest price during the day.]**

**C = Close [Closing price.]**

**V = Volume [Amount of trading activity.]**

Example:

| Date  | Open | High | Low | Close | Volume |
|-------|------|------|-----|-------|--------|
| Day 1 | 100  | 105  | 98  | 103   | 1M     |

## 24. What is SMA20?

SMA = Simple Moving Average.

SMA20 means: Average closing price of the previous 20 trading days.

Formula:

$$SMA_{20} = \frac{Close_1 + Close_2 + \dots + Close_{20}}{20}$$

Example:

20-day prices

↓

Add them

↓

Divide by 20

↓

SMA20

It helps represent the recent trend.

## 25. SMA50

SMA50=sum of previous 50 closes

$$SMA_{50} = \frac{\text{sum of previous 50 closes}}{50}$$

SMA20 reacts faster.

SMA50 is slower.

Conceptually:

SMA20 → shorter-term trend

SMA50 → somewhat longer-term trend

## 26. RSI14

RSI = Relative Strength Index.

It measures recent upward vs downward price momentum.

Common calculation:

### Step 1

Calculate gains and losses.

Price increases → Gain

Price decreases → Loss

### Step 2

Calculate average gain and average loss over 14 periods.

### Step 3

RS=Average Gain/Average Loss

### Step 4

RSI=100-100/1+RS

RSI ranges from approximately:

```
0 → 100
```

For our project:

RSI provides an additional momentum-related feature to the LSTM.

## 27. MACD

MACD = Moving Average Convergence Divergence.

It compares exponential moving averages.

The standard configuration is commonly:

```
EMA12
EMA26
Signal = EMA9 of MACD
```

Main calculation:

$$MACD = EMA_{12} - EMA_{26}$$

Then:

$$Signal = EMA_9(MACD)$$

MACD gives information related to trend and momentum.

## 28. Why use technical indicators?

Because instead of giving LSTM only:

```
Open
High
Low
Close
Volume
```

we also provide derived information about:

```
Trend → SMA

Momentum → RSI

Trend/momentum relationship → MACD
```

So:

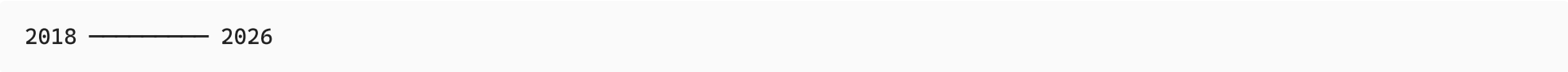


But every feature must be calculated using **past/current information only**, never future information.

## 29. What is Walk-Forward Validation?

This is extremely important.

Imagine our data:



We can't randomly mix the dates.

Instead:

Fold 1



Fold 2



Fold 3

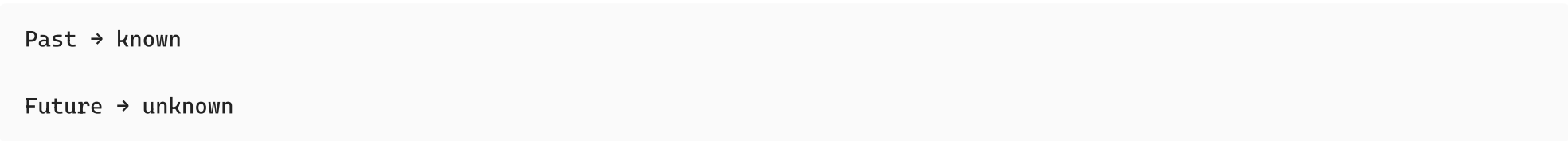


The training window **expands forward in time**.

30. Why do we need Walk-Forward Validation?

Because we are forecasting the future.

In real life:



Random splitting can accidentally allow information from different time periods to mix.

Walk-forward validation simulates the real forecasting situation:

Train using the past → predict the future → move forward → train again → predict the next future period.

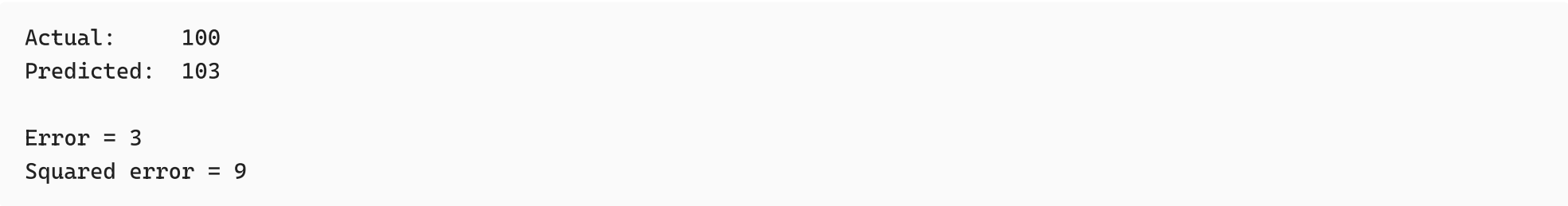
31. What is MSE?

MSE = Mean Squared Error.

Formula:

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Example:



For multiple predictions:

Calculate errors

↓

Square errors

↓

Average them

Our LSTM uses **MSE as the training loss** according to the PRD.

## 32. What is RMSE?

RMSE = Root Mean Squared Error.

$$RMSE = \sqrt{MSE}$$

Why take the square root?

Because it puts the result back into the same unit as the target.

If stock price is measured in:

₹

RMSE is also roughly interpretable in:

₹

**Lower RMSE = smaller prediction errors.**

Your PRD requires RMSE as the main price-forecasting metric.

## 33. MSE vs RMSE

Simple:

MSE

↓

Squared error

RMSE

↓

$\sqrt{MSE}$

↓

Same scale as target

We use:

**MSE**

For **training the LSTM**.

**RMSE**

For **evaluating/reporting forecasting performance**.

That's a very good viva answer.

## 34. What is MAE?

MAE = Mean Absolute Error.

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

Example:

| Actual | Prediction    |
|--------|---------------|
| 100    | 103 → error 3 |
| 110    | 108 → error 2 |
| 120    | 124 → error 4 |

MAE:

$(3+2+4)/3=3$

So average absolute error = **3**.

Your PRD recommends MAE as an additional evaluation metric.

### 35. RMSE vs MAE

Remember:

| Metric | Meaning                |
|--------|------------------------|
| MSE    | Average squared error  |
| RMSE   | Square root of MSE     |
| MAE    | Average absolute error |

#### Why use RMSE?

It penalizes large errors more strongly because errors are squared.

#### Why use MAE?

It is easy to interpret and less sensitive to extreme errors.

### 36. What is Directional Accuracy?

We aren't only interested in:

"How close was my predicted price?"

We also want:

"Did I correctly predict whether the price moved up or down?"

Example:

|                        |
|------------------------|
| Current price = ₹100   |
| Actual tomorrow = ₹110 |
| Prediction = ₹105      |

Price isn't exact.

But:

|                          |
|--------------------------|
| Actual direction = UP    |
| Predicted direction = UP |

So direction is correct.

## 37. How calculate Directional Accuracy?

For each prediction:

Actual return

$$ActualReturn = \frac{ActualClose - CurrentClose}{CurrentClose}$$

Predicted return

$$PredictedReturn = \frac{PredictedClose - CurrentClose}{CurrentClose}$$

Then compare their directions.

If:

Actual return > 0  
AND  
Predicted return > 0

→ Correct.

If:

Actual return < 0  
AND  
Predicted return < 0

→ Correct.

Otherwise:

→ Incorrect.

Then:

$$DirectionalAccuracy = \frac{CorrectDirectionPredictions}{TotalPredictions} \times 100$$

Your project specifically requires directional accuracy in addition to price RMSE.

## 38. Why not just use precision, recall and accuracy?

Example:

₹523.45

That's a regression problem.

Therefore:

### For price prediction

Use:

RMSE  
MAE  
MSE

### For direction classification

Use:

- Directional Accuracy
- Confusion Matrix
- Precision
- Recall
- F1

## 39. Should we calculate Precision and Recall?

You **can**, especially because your trend output has three classes:

- Bullish
- Neutral
- Bearish

Then precision/recall become meaningful.

But they are **not the primary metrics specified by your PRD**.

Accuracy

$$Accuracy = \frac{CorrectPredictions}{TotalPredictions}$$

### Precision

For Bullish, for example:

Of all predictions we called Bullish, how many were actually Bullish?

Precision=

$$Precision = \frac{TP}{TP + FP}$$

### Recall

Of all actual Bullish cases, how many did we correctly identify?

Recall=

$$Recall = \frac{TP}{TP + FN}$$

## 40. Why confusion matrix?

Because you have three classes.

|              | Actual |         |      |
|--------------|--------|---------|------|
|              | Bull   | Neutral | Bear |
| Pred Bull    |        |         |      |
| Pred Neutral |        |         |      |
| Pred Bear    |        |         |      |

It shows exactly where the model is making mistakes.

For example:

Actual Bullish → Predicted Neutral

is different from:

Actual Bullish → Predicted Bearish

The confusion matrix shows this.

## 41. Your three trend signals

This is directly specified in your PRD.

First calculate:

$$PredictedReturn = \frac{PredictedClose - CurrentClose}{CurrentClose} \times 100$$

Then:

### Bullish

$$Predicted\ Return > +0.5\%$$

Example:

Current = ₹100  
Predicted = ₹101

Return = +1%

→ Bullish

### Neutral

$$-0.5\% \leq Return \leq +0.5\%$$

Example:

Current = ₹100  
Predicted = ₹100.2

Return = +0.2%

→ Neutral

### Bearish

$$Return < -0.5\%$$

Example:

Current = ₹100  
Predicted = ₹98.5

Return = -1.5%

→ Bearish

These exact thresholds are specified in your model card.

## 42. Why ±0.5%?

Because very small movements can be noise.

Without a neutral zone:

```
+0.01%
→ Bullish

-0.01%
→ Bearish
```

That would generate too many signals.

Your PRD explicitly explains the trade-off:

```
Threshold too small
→ too many signals

Threshold too large
→ too many Neutral signals
```

And importantly:

**$\pm 0.5\%$  is your project's initial policy, not a universal financial truth.**

---

## 43. What is Backtesting?

Backtesting means:

**Pretending we had used our model in the past and checking what the strategy would have done.**

Example:

```
Historical data
  ↓
Model prediction
  ↓
Generate signal
  ↓
Simulate trade
  ↓
Calculate return
```

---

## 44. Why do we need Backtesting?

Because:

**RMSE tells us:**

How accurately did we predict price?

**Directional Accuracy tells us:**

How often did we predict direction correctly?

But neither directly answers:

"What would happen if we used these signals in a trading strategy?"

Backtesting gives us that additional perspective.

---

## 45. What should our backtest calculate?

At minimum:

```
Initial Capital
Final Capital
```

Total Return  
Number of Trades  
Win Rate  
Maximum Drawdown  
Sharpe Ratio

And we should define:

Transaction costs  
Slippage  
Execution assumptions

because historical backtesting can differ from real trading. Your PRD explicitly highlights this limitation.

---

## 46. How do we compare LSTM with baseline?

This is extremely important.

Suppose our test data has:

1000 predictions

We generate:

### Naive

Prediction = Today's Close

### LSTM

Prediction = LSTM output

Then calculate the same metrics.

Example:

| Model | RMSE | MAE | Directional Accuracy |
|-------|------|-----|----------------------|
| Naive | 10.2 | 7.8 | 51%                  |
| LSTM  | 8.6  | 6.4 | 56%                  |

Then we can say:

"On this evaluation period, LSTM produced lower RMSE and higher directional accuracy than the naive baseline."

That is a **descriptive result**, not a guarantee that LSTM will always outperform.

---

## 47. How should we compare them fairly?

Very important:

Both models must use:

Same dataset  
Same test periods  
Same evaluation folds  
Same target  
Same metric calculations

Don't do:

```
Naive → 2022 data

LSTM → 2025 data
```

That isn't a fair comparison.

## 48. Walk-forward comparison

Our evaluation should look conceptually like:



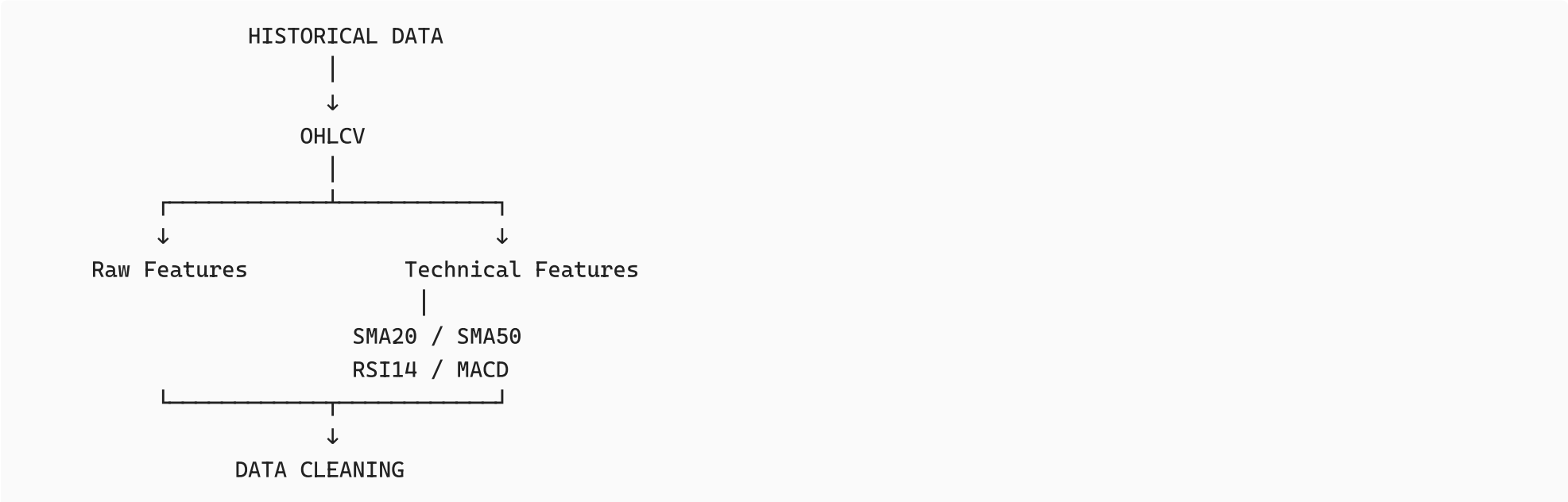
Then report:

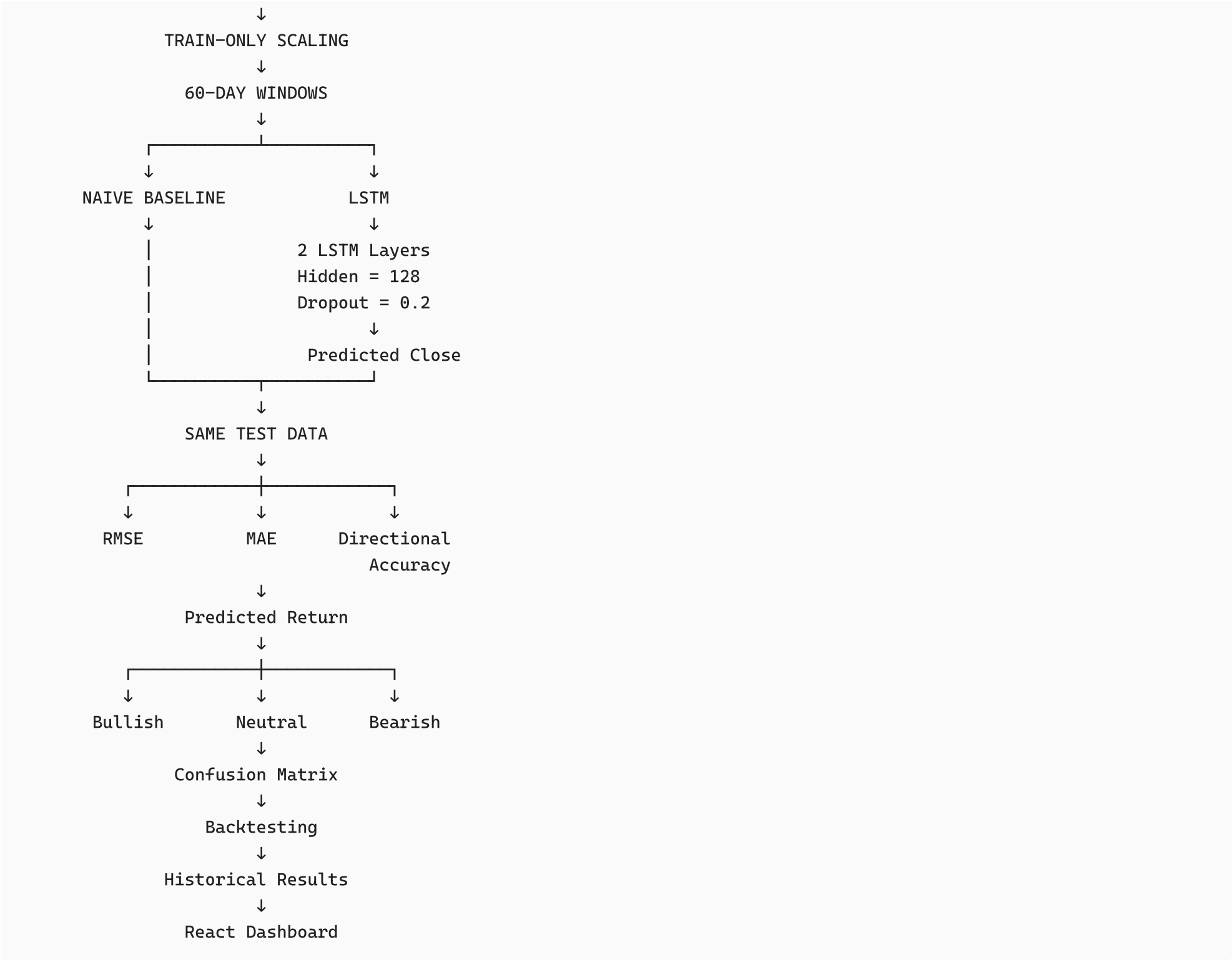
```
Fold 1
Fold 2
Fold 3
Fold 4
Fold 5
-----
Overall summary
```

Your PRD requires metrics to be evaluated per walk-forward fold against the naive baseline.

## 49. The entire project logic

Now put everything together:





The PRD's complete lifecycle is essentially **Data** → **Cleaning** → **EDA** → **Features** → **Sequence Generation** → **Training** → **Validation** → **Evaluation** → **Forecasting** → **Backtesting** → **API** → **Frontend** → **Deployment**.

## 50. The most important "WHY" answers

If your professor asks **"Why?"**, remember this table:

| Question            | Simple answer                                                     |
|---------------------|-------------------------------------------------------------------|
| Why LSTM?           | Sequential data + temporal dependencies                           |
| Why not only Naive? | Naive is too simple; we want to test whether LSTM adds value      |
| Why Naive baseline? | Simple, transparent reference point                               |
| Why ARIMA?          | Valid statistical alternative, can be added as secondary baseline |
| Why not SARIMA?     | No primary focus on explicit seasonality                          |
| Why not ARIMAX?     | We aren't using external variables in MVP                         |
| Why 60 days?        | Good initial recent-history window; test 30/60/90                 |
| Why 2 LSTM layers?  | Learn hierarchical temporal representations; tune if needed       |
| Why 3 gates?        | Control forgetting, remembering and output                        |
| Why cell state?     | Long-term memory                                                  |
| Why hidden state?   | Current useful representation/output                              |
| Why OHLCV?          | Core historical market information                                |
| Why SMA?            | Trend information                                                 |
| Why RSI?            | Momentum information                                              |
| Why MACD?           | Trend/momentum information                                        |
| Why scaling?        | Helps neural-network training                                     |

| Question                  | Simple answer                                             |
|---------------------------|-----------------------------------------------------------|
| Why train-only scaling?   | Prevent data leakage                                      |
| Why walk-forward?         | Future must always remain unseen                          |
| Why RMSE?                 | Measures price prediction error; penalizes large errors   |
| Why MAE?                  | Easy-to-understand average error                          |
| Why Directional Accuracy? | Price closeness isn't the same as getting direction right |
| Why confusion matrix?     | Shows which trend classes are confused                    |
| Why $\pm 0.5\%$ ?         | Ignore very small movements/noise                         |
| Why backtesting?          | Evaluate historical strategy behavior                     |
| Why compare baseline?     | Determine whether LSTM actually adds value                |

## 🔥 51. One thing you should change in how you explain the project

Don't say:

"We selected LSTM because LSTM is the best model for stock prediction."

That's too strong and scientifically incorrect.

Say:

"We selected LSTM as the primary model because the project focuses on deep-learning-based sequential forecasting. We use a naive baseline to establish a simple reference, and we evaluate whether LSTM actually provides improvement through leakage-safe walk-forward testing."

This is much stronger.

Your PRD itself says the initial settings are **experimental settings, not final measured results**, and that hyperparameters should be tuned using training/validation data.

## 🎯 52. Your 10-minute viva cheat sheet

If you remember only this, you're covered:

### Problem

Predict tomorrow's closing price using the previous 60 trading days.

### Input

OHLCV + SMA20 + SMA50 + RSI14 + MACD

### Model

2-layer LSTM  
Hidden = 128  
Dropout = 0.2

### Why LSTM

Sequential data + temporal dependencies.

### Why baseline

To determine whether the complex model actually improves over a simple method.

### Naive

Tomorrow = Today

## 60 days

Initial configurable lookback; compare 30/60/90 during experimentation.

## Gates

|                          |  |
|--------------------------|--|
| Forget → what to remove  |  |
| Input → what to remember |  |
| Output → what to expose  |  |

## Cell state

Long-term memory.

## Hidden state

Current useful representation/output.

## Validation

Expanding walk-forward, because future data must not be used to train the past.

## Metrics

|                                                   |
|---------------------------------------------------|
| MSE → training loss                               |
| RMSE → price error                                |
| MAE → average absolute error                      |
| Directional Accuracy → direction correctness      |
| Confusion Matrix → Bullish/Neutral/Bearish errors |

## Trend

|                          |
|--------------------------|
| > +0.5% → Bullish        |
| -0.5% to +0.5% → Neutral |
| < -0.5% → Bearish        |

## Backtesting

Simulate historical use of signals and calculate return, drawdown, Sharpe, etc.

## Final comparison

|                                   |
|-----------------------------------|
| Naive vs LSTM                     |
| ↓                                 |
| Same folds                        |
| ↓                                 |
| Same test periods                 |
| ↓                                 |
| RMSE + MAE + Directional Accuracy |
| ↓                                 |
| Backtest                          |

And the most important principle:

**We are not trying to prove that LSTM can predict the stock market perfectly. We are testing whether LSTM provides useful predictive information beyond a simple baseline under a fair, leakage-free, chronological evaluation.**

That is the correct research mindset for your project.